

Backend para AURA Sentinel

Decisión estratégica: Backend híbrido, open-source y modular

El enfoque más competitivo y sostenible para AURA Sentinel es adoptar una arquitectura backend basada en software libre, modular y enfocado a "offline-first", para cumplir con las etapas del concurso y asegurar migración y expansión sin fricciones.

1. Corto plazo (Concurso/MVP)

Tecnología recomendada: Appwrite

- Ventajas:
 - Plataforma open-source, fácilmente auto-hospedable o desplegable en nubes públicas/privadas.
 - Integración nativa con Flutter y SDKs robustos para mobile.
 - Módulos de autenticación, base de datos, funciones serverless y almacenamiento integrado.
 - Sincronización básica offline-local y control granular de datos y usuarios.
 - Cumplimiento sencillo de la LFPDPPP: puedes decidir la localización, resguardo y ciclo de vida de los datos desde el inicio.
- Motivo estratégico:
 - Permite iterar rápidamente el MVP, mantener bajo costo y controlar completamente los datos sensibles de usuarios, pieza esencial para competencia y confianza.

2. Mediano plazo (Piloto real / primeros 1,000-10,000 usuarios)

Estrategia:

- Migración transparente a infraestructura más robusta:
 - Escalamiento a servidores dedicados (cloud propia, servidores nacionales) o proveedores como Back4App.

- Conservas lógica de frontend Flutter y migras backend sin reescribir ni perder lógica/módulos previos.
- Facilita integración de APIs avanzadas para IA (e.g., Grok/Claude) con funciones tipo proxy o microservicio, elevando capacidades analíticas conforme crece el uso real.

3. Largo plazo (escala nacional/global)

Estrategia:

- Posibilidad de migración a stacks enterprise (AWS, GCP, Azure) o adopción de capas avanzadas tipo Couchbase/Supabase para sincronización distribuida.
- Arquitectura modular desde inicio: soporta DevOps, integración continua, plugins de IA y paneles administrativos para actores públicos (911, protección civil).
- El stack nunca te ata a un proveedor específico y permite implementar nuevas normativas, métodos de cifrado o analítica avanzada sin rehacer la app mobile.

Elección por fases – Resumen ilustrativo

Fase	Backend sugerido	Motivo	Escalabilidad
MVP/Concurso	Appwrite	Control, facilidad, privacidad, costo cero	Migrable
Piloto real	Appwrite/Back4App	Escala, soporte, integración APIs externas	Totalmente libre
Nacional	AWS, Couchbase, etc.	Alta disponibilidad, multi-región	Modular

Justificación y links de referencia

- No te “encadenas” a proveedores cerrados como Firebase.
- Cumples legislación y requisitos técnicos desde el día 1 (LFPDPPP).
- Puedes empezar chico y crecer rápido, sin refactorizaciones masivas.
- Infraestructura lista para ambientes DevOps, migración y paneles IA.

Recursos:

- appwrite.io - Comparativa BaaS 2025
- [Alternativas a Firebase \(Back4App\)](#)

Conclusión:

Esta planeación te da flexibilidad, control, cumplimiento legal y velocidad, y prepara tu proyecto para competir, impactar y escalar profesionalmente, alineada al perfil y objetivos de AURA Sentinel.

Diseño de Arquitectura, APIs y Módulos Backend – AURA Sentinel

1. Enfoque Estratégico

1. El backend de AURA Sentinel se diseña bajo una arquitectura híbrida, modular y open-source, alineada con la filosofía offline-first y los principios de seguridad, escalabilidad y soberanía de datos.
El objetivo es garantizar la continuidad operativa en entornos de baja conectividad, permitir una migración sin fricciones y cumplir con las normativas de protección de datos desde las primeras etapas del proyecto.

Partes Integradas Recientemente al Documento

1.2 Paso a Paso de Implementación Detallada (Flutter & Appwrite)

Esta es la adición más grande y se enfoca en cómo la arquitectura se traduce en código, implementando el principio **Offline-First**.

- **Instalación de Dependencias:** Se especifica el uso de `sqflite`, `path`, `appwrite`, y `connectivity_plus`.
- **Servicio de Base de Datos Local (`db_service.dart`):**
 - Se define la inicialización de la base de datos **SQLite** (`aura_sentinel.db`).

- Se crearon los esquemas de las tablas que operarán sin conexión: **usuarios, ficha_medica, contactos_emergencia y emergencias.**
- **Configuración de Appwrite (`appwrite_service.dart`):** Se establece la conexión con el **Endpoint** y el **Project ID** del backend en la nube.
- **Mecanismo de Sincronización:**
 - Se incluye la función **`syncUsuario`** para subir datos específicos de SQLite a Appwrite.
 - Se incluye la función **`checkAndSync`** que usa **`connectivity_plus`** para detectar la red e iniciar la subida de datos pendientes.
- **Inicialización en la App:** Se muestra cómo el **`main.dart`** inicializa la base de datos local y comienza el proceso de sincronización al arrancar.

Decisión estratégica

Tipo: Backend híbrido modular, autosuficiente y portable.

Principios: Control total, escalabilidad por fases y cumplimiento legal (LFPDPPP).

Ventaja clave: Crecimiento progresivo sin necesidad de refactorización masiva o dependencia de proveedores cerrados.

2. Arquitectura General

2.1. Vista por capas

La arquitectura propuesta se compone de cuatro capas principales:

Capa de Dispositivo (Frontend/Edge)

- Aplicación móvil desarrollada en Flutter, con integración directa a modelos TFLite
- Procesamiento local de IA: clasificación de emergencias, guía emocional, activación por voz
- Sincronización diferida con backend cuando la conexión es estable

Capa de Procesamiento Backend (Appwrite Stack)

- Núcleo de servicios alojado en Appwrite
- Módulos: autenticación, base de datos, funciones serverless, almacenamiento de fichas y logs
- API REST/GraphQL nativa
- Totalmente auto-hospedable y con control de localización de datos

Capa Híbrida de Integración

- Conexión con APIs externas:
Servicios emergencia (911), SMS redundante (Twilio), IA Proxy para guía avanzada

Capa de Datos y Escalabilidad

- Gestión de datos distribuida con monitoreo continuo
- Contenedores y CICD para despliegue continuo

3. Módulos Backend Principales

Módulo	Tecnología	Función	Fase
API Gateway	Appwrite / Nginx	Control de acceso	Todas
Auth / Users	Appwrite Auth	Identidades y roles	MVP
Database Core	Appwrite DB	Fichas médicas, logs, ubicación	MVP
Serverless Functions	Node.js / Python	Microservicios de IA y alertas	MVP
Sync Manager	Appwrite Functions	Offline → Cloud	Piloto
Emergency Notifier	Twilio / API 911	Envío de alertas	Piloto
IA Proxy Service	FastAPI	Instrucciones médicas avanzadas	Piloto

Admin Panel	software	Gestión para instituciones	Nacional
-------------	----------	----------------------------	----------

4. Diseño de APIs

4.1. Nivel interno (Appwrite / Core)

- /account/login
- /account/register
- /database/{collection}/list
- /functions/{id}/execute

4.2. Nivel AURA Sentinel

Endpoint	Método	Descripción
/emergency/report	POST	Reporte de emergencia
/geo/refuges	GET	Refugios cercanos
/user/ficha	GET / PUT	Ficha médica
/alert/911	POST	Notificación a servicios
/sync	POST	Sincronización offline
/ai/guide	POST	Guía médica IA

5. Escalabilidad por Fases

Fase	Backend	Enfoque	Beneficio
Corto plazo (MVP)	Appwrite	Privacidad y bajo costo	Iteración rápida
Mediano plazo	Appwrite + APIs externas	Integraciones	Mayor alcance
Largo plazo	Multi-región cloud	Alta disponibilidad	Escalabilidad total

6. Seguridad y Cumplimiento

- Cifrado AES-256 y TLS 1.3
- JWT y control granular de permisos
- Cumple LFPDPPP y localización nacional de datos

7. Resumen Visual Conceptual

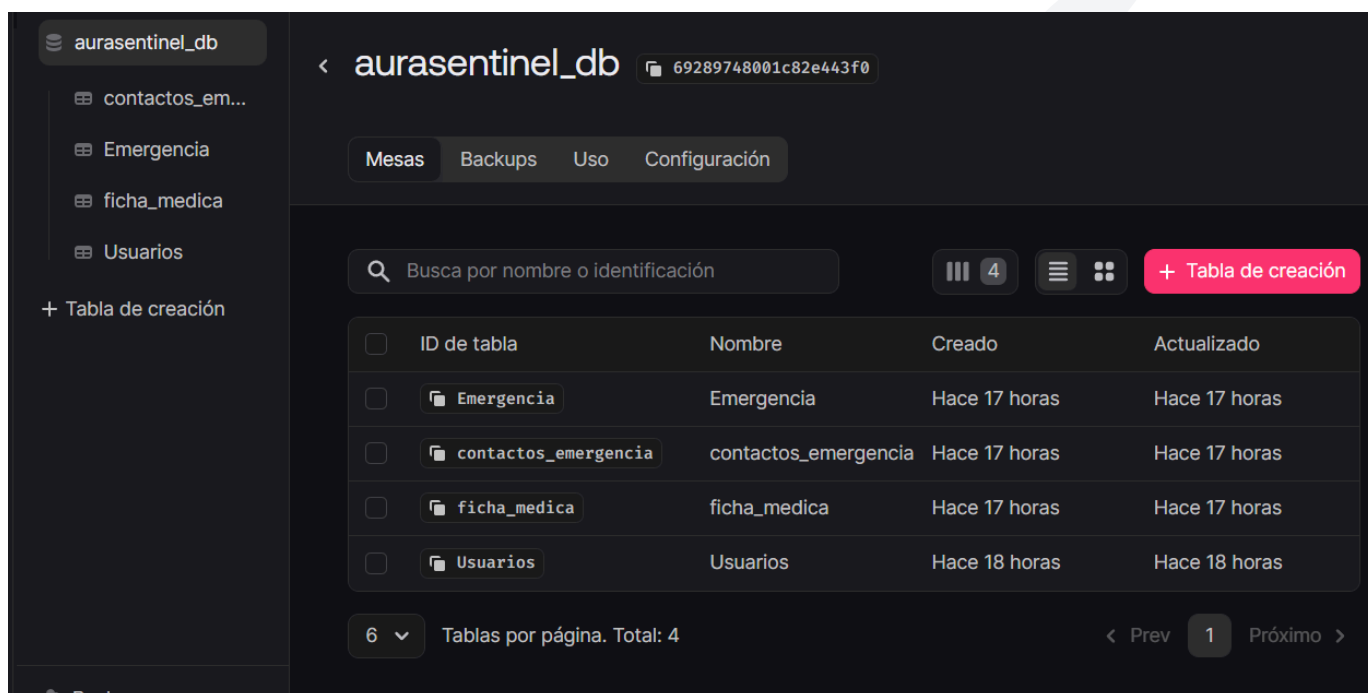


8. Integración con Appwrite (Nueva sección añadida)

Configuración de Backend

Se habilita la conexión Flutter ↔ Appwrite mediante:

- Proyecto: **AURA Sentinel**
- Método Auth: Email + Password
- Base de datos: colección **ficha_medica**
- Roles: permisos solo para usuario autenticado



- Endpoint y Project ID configurados en Flutter

Atributos de ficha_medica:

- nombre
- edad
- peso
- género
- grupo_sanguíneo

- alergias
- condición_médica_crónica
- toma_medicamentos
- seguro_médico

Integración Flutter – SDK Appwrite:

```
final client = Client()

  .setEndpoint('https://cloud.appwrite.io/v1')

  .setProject('PROJECT-ID');

final account = Account(client);
```

Registro y Login:

```
await account.create(

  userId: ID.unique(),

  email: email,

  password: password,

);

await account.createEmailPasswordSession(email: email, password:
password);
```

Guardar la ficha médica:

```
await Databases(client).createDocument(

  databaseId: 'fichaDB',

  collectionId: 'ficha_medica',

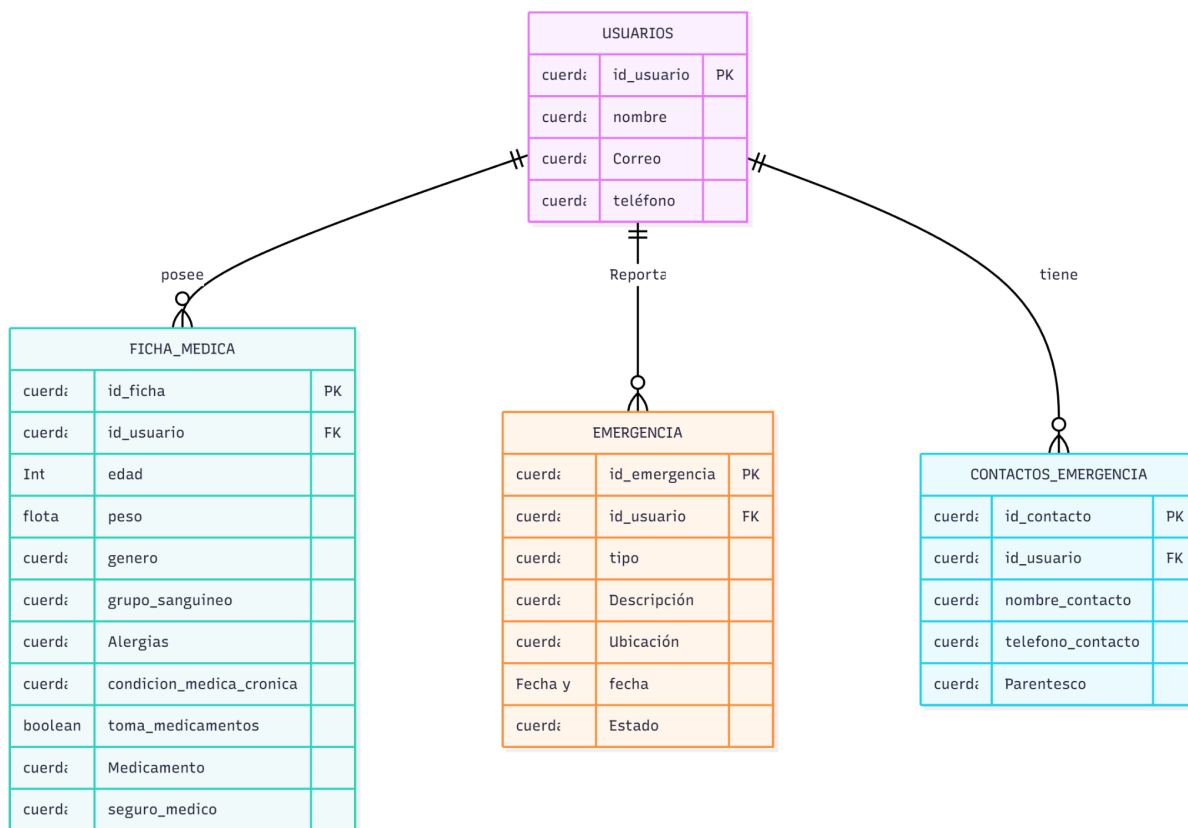
  documentId: ID.unique(),

  data: {...},
```

);

- ♦ Con esto se completa la **Fase 1 funcional del backend**

ENTIDAD - RELACIÓN



9. Conclusión

El backend de AURA Sentinel garantiza:

- Resiliencia y operación en emergencias reales
- Protección total de datos sensibles
- Integración IA + servicios prioritarios
- Expansión nacional sin reconstrucción del sistema

AURA Sentinel es una plataforma confiable, escalable y vital para salvar vidas en situaciones críticas.